



Universidad de Jaén

Tema 1.1 Introducción

Arquitectura y conceptos básicos en aplicaciones web

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

v 1.6.5

Este obra está bajo una [Licencia Creative Commons](#)
Atribución-CompartirIgual 4.0 Internacional.

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Objetivos

- Identificar los elementos principales de la **arquitectura** de una aplicación web
- Conocer la **evolución histórica** de las tecnologías web más relevantes
- Conocer las diferencias principales entre **metodología** tradicional y actual en el desarrollo de aplicaciones web
- Tener una **visión general de las tecnologías** asociadas a los elementos principales de la arquitectura de un aplicación web
- Familiarizarse en general con términos, tecnologías y **conceptos relacionados** con el desarrollo de aplicaciones web



¿Qué es una aplicación web?

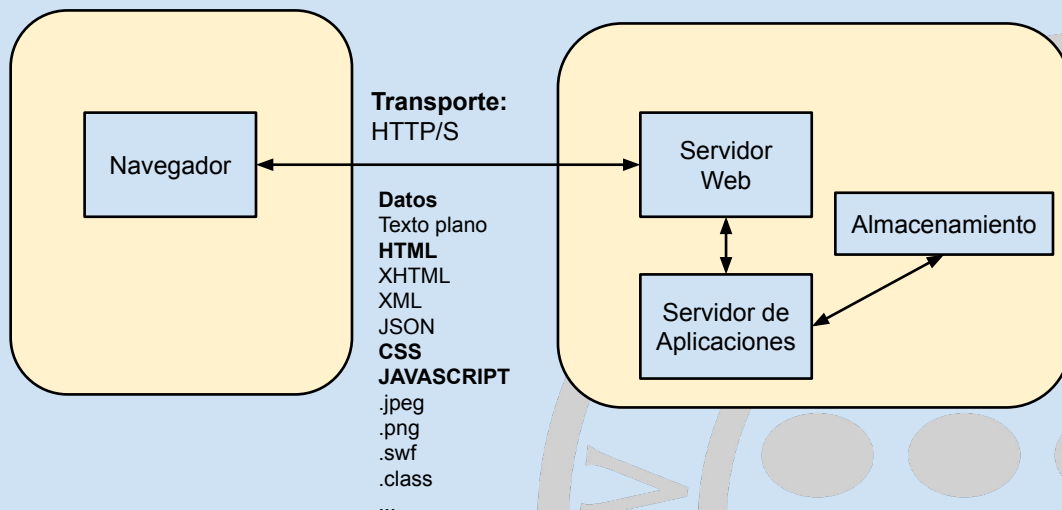
Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Arquitectura básica de una aplicación web

Procesamiento en el lado del cliente

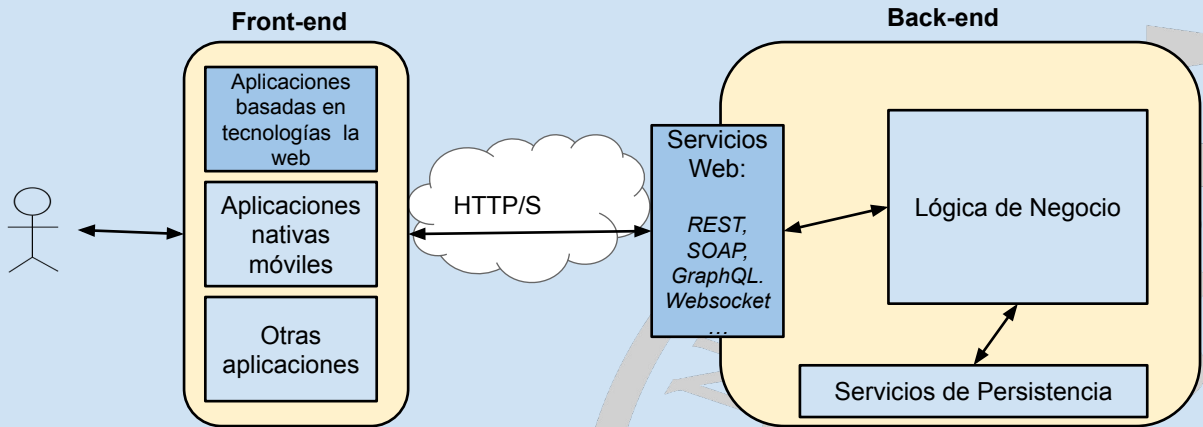
Procesamiento en el lado del Servidor



Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Aplicaciones web en el ámbito empresarial



5

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Inicios de las tecnologías web

- **Presentación de contenidos: HTML/CSS**
 - 1989, Tim Berners-Lee y Robert Cailliau presentan un proyecto para intercambiar información hipertextual (**HTML**) a través de Internet usando el protocolo HTTP. <http://info.cern.ch/>
 - 1996, **CSS1** como lenguaje de estilos
- **Lenguajes de programación en el servidor**
 - 1993, especificación para llamar a "scripts" en el servidor. Common Gateway Interfaz (**CGI**)
 - 1995, **PHP** (*Personal Home Page/PHP Hypertext Preprocessor*) para construir scripts en el servidor que generen páginas web de forma dinámica
 - 1997, Microsoft Active Server Pages (**ASP**); Sun Java **Servlets**; plataforma Sun Java 2 Enterprise Edition (**J2EE**)
- **Lenguajes de programación en el cliente**
 - 1996, **Javascript** como lenguaje de "scripts" en cliente Especificación **Applet Java**; Macromedia **Flash Player 1.0** (ActionScript); Microsoft **ActiveX**, JScript (internet explorer) como orígenes de las *Rich Interfaces Applications* (RIA)

6

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Situación actual de las principales tecnologías web

- **Presentación de contenidos: HTML/CSS**

- **HTML 5** para creación y organización semántica de contenidos. xhtml y otras tecnologías basadas en xml, e.g. xpath, xquery,... como herramientas para verificación formal de contenidos
- **CSS**, especificaciones modulares con diferentes niveles, e.g. CSS2 Grid

- **Lenguajes de programación en el servidor**

- Utilizados normalmente junto a frameworks de programación
- PHP (Symfony/Laravel), Java (JakartaEE/Spring), C# (.NET), Javascript (Nodejs), Python (Django), Ruby (Rails), Go...

- **Lenguajes de programación en el cliente**

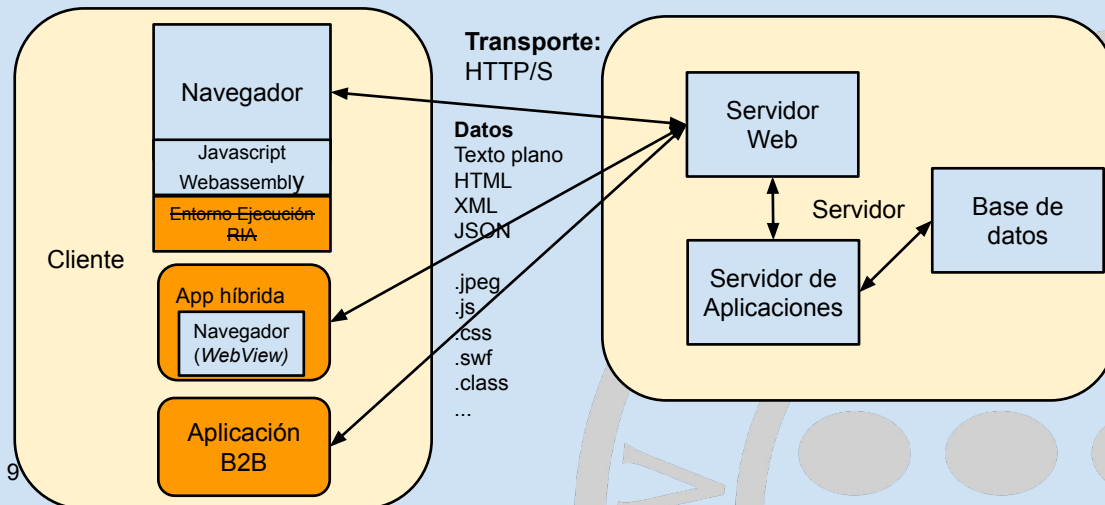
- **Soporte nativo en navegador**
 - Javascript con **frameworks de programación**: React, Vue, Angular... y meta-lenguajes: CoffeeScript, Typescript, Dart,...
 - **Webassembly** como punto de entrada a otros lenguajes de programación, e.g. c++
- Plugins del navegador (en desuso):
 - ~~Applets (Java); Macromedia Flash (ActionScript); Microsoft ActiveX; Silverlight (.net)~~
- **Aplicaciones híbridas**
 - Meteor, Electron, Ionic, ReactNative, Flutter, Cordova, Framework 7...

¿Cómo se construye una aplicación web?

Arquitectura básica de una aplicación web

Procesamiento en el lado del cliente

Procesamiento en el lado del Servidor



Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Evolución de las metodologías de desarrollo web

● Metodología tradicional:

- **Mezcla de diferentes lenguajes en un mismo fichero:** html, css, javascript, php, ...
- **URLs asociadas a nombres de ficheros** en servidor. Enlaces a ficheros físicos.
- **Arquitectura monolítica:** lógica de control y acceso a datos mezclada
- **Aplicación web centrada en el servidor:**
 - Procesamiento de información y generación dinámica de contenidos HTML en servidor para envío al cliente. *Server-Side Scripting/Rendering (SSR)*
 - El cliente visualiza HTML, gestiona enlaces (GET) y envíos de formularios (POST)
- **Diseño orientado a ordenador de escritorio**
 - versión específica para dispositivos móviles

Evolución de las metodologías de desarrollo web

- **Metodologías actuales:**

- **Aplicación web centrada en el cliente**

- Servidor como **servicio web** (*REST, SOAP, GraphQL...*) de intercambio de datos estructurados (*JSON, XML,...*)
- Modificación dinámica de vistas HTML en cliente (*javascript, ajax*)- *Client-Side Rendering (CSR)*... aunque se puede usar SSR en situaciones específicas, e.g. SEO
- Lógica de control en cliente

- **Diseño orientado a dispositivos móviles (*Mobile First*)**

- Diseño adaptativo (*responsive design*) de interfaz
- **Aplicaciones Simple Page Applications, SPA y Progressive Web Applications, PWA**
- **Aplicaciones híbridas** basadas en html5/css/javascript (Ionic, React native, Cordova...)

- **Arquitecturas Modulares**

- Basadas en **Modelo-Vista-Controlador en servidor y en cliente**
- **Vistas** en html con uso de plantillas para contenidos dinámicos
 - Hojas de estilo vinculadas a vistas (archivos .css)
 - Módulos javascript independientes (archivos .js)
- Módulos con lógica de control (**Controladores**)
- Módulos para acceso a datos o lógica de negocio (**Modelo**)
- **Enrutamiento de URLs** a módulos de procesamiento

11

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Comunicación con el Servidor

El protocolo HTTP



Universal Resource Locator (URL)

¿Cómo identificar cualquier recurso en la web?

protocolo://usuario:clave@host:port/ruta/recurso/ruta_adicional?var1=data&var2=data#elemento

Ejemplos

<https://jrbalsas:12345@www.ujaen.es:80/info/guia.do/informatica?cod=13312007#evaluacion>

<ftp://anonymous:anonymous@ftp.ujaen.es/pub/descargas/cambiogrupo.pdf>

Elemento	Descripción	Valor en ejemplo
protocolo	Indica al cliente cómo contactar con el servidor. e.g. http, https, ftp, etc.	https
usuario	Usuario que se conecta al recurso	jrbalsas
clave	Clave de acceso para autenticación	12345
servidor:puerto	Nombre o dirección IP y puerto tcp del servidor	www.ujaen.es y puerto 80
ruta	Ruta en el servidor para acceder al recurso. Cada segmento puede tener parámetros separados por ;	info info;curso=1718
recurso (<i>script</i>)	Recurso solicitado, e.g. archivo, programa, funcionalidad...	guia.do
ruta_adicional (<i>path info</i>)	fragmento de ruta posterior al recurso hasta el carácter ?	informatica
consulta (<i>query</i>)	cadena posterior al carácter ? Pares <i>param=valor</i> separados por & o ;	cod=13312007
fragmento (hashtag)	identificador de zona después de # (atributo id en etiqueta html)	evaluacion

13

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



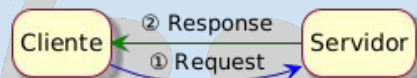
Comunicación con el servidor

• Protocolo HTTP

- Versión 1.1 (RFC2616 de Junio 1999), Versión 2.0 (2015, RFC7540), Versión 3.0 (2022, RFC9114)
- Originalmente:
 - Orientado a **transacciones** (petición-respuesta)
 - **Sin estado**: transacciones independientes (la conexión TCP se desconecta)
- Se han ido introduciendo **mejoras** como las conexiones persistentes o más recientemente los websockets
- En **HTTP/2**, compresión encabezados, multiplexación de peticiones, peticiones priorizadas y envíos desde servidor
- **HTTP/3**, conexiones multiplexadas y uso de UDP (Quic)

• Fases de una transacción HTTP:

- Petición al servidor (**Request**)
- Respuesta del servidor (**Response**)



14

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Protocolo HTTP

Petición (Request) HTTP

Formato de texto plano:

<tipo petición> <recurso solicitado> <versión HTTP>

Parámetros de cabecera automáticos (nombre servidor, tipo de datos, etc.) y personalizados por usuario

Línea en blanco

Cuerpo del mensaje

Ejemplos:

```
GET /index.html HTTP/1.1
```

```
Accept: text/html
```

```
Host: www.ujaen.es
```

```
<sin datos en el cuerpo>
```

```
POST /login.php HTTP/1.1
```

```
Accept: text/html
```

```
Host: www.ujaen.es
```

¿Es seguro
enviar una clave
en texto plano?

```
user=jrbalsas&password=dawforever
```

15

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Protocolo HTTP

Respuesta (Response)

Formato

- Protocolo y código de estado
- Encabezados
 - automáticos
 - personalizados
- Línea en blanco
- Cuerpo del mensaje con datos de respuesta

```
HTTP/1.1 200 OK
```

```
Date: Sun, 27 Jan 2013 12:16:40 GMT
```

```
Server: Apache
```

```
Last-Modified: Sun, 27 Jan 2013  
12:15:08 GMT
```

```
ETag: "6336-4d4441c822b00"
```

```
Accept-Ranges: bytes
```

```
Content-Length: 25398
```

```
Cache-Control: no-store, no-cache,  
must-revalidate, post-check=0,  
pre-check=0
```

```
Expires: Sun, 19 Nov 1978 05:00:00 GMT
```

```
Connection: close
```

```
Content-Type: text/html; charset=utf-8
```

```
<!DOCTYPE html ><html><head>...
```

16

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Códigos de estado HTTP

<http://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml>

Categoría	Tipo	Ejemplo
1xx	Información	100 Continuar con el envío de información adicional
2xx	Éxito	200 Ok
3xx	Redirección	307 Redirección temporal
4xx	Error del cliente	400 Solicitud incorrecta 404 Recurso no encontrado
5xx	Error del servidor	500 Error interno



17

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Tipos de peticiones HTTP

- **GET** Petición de información al servidor
 - <http://www.ujaen.es/guias.do?codigo=13312007>
- **POST** Envío de información (alta)
 - La información se adjunta en el campo de datos HTTP (body)
- **PUT** Modificación completa de un recurso específico
- **DELETE** Borrado de un recurso específico
- **PATCH** Modificación parcial de un recurso específico
- **OPTIONS** Devuelve los tipos de peticiones soportados por un recurso, e.g GET, POST, etc.
- **HEAD** Similar a GET pero sin datos de respuesta (solo cabecera)
- **CONNECT** Comprobación de conexión con el servidor
- **TRACE** Devuelve la información enviada con fines de diagnóstico

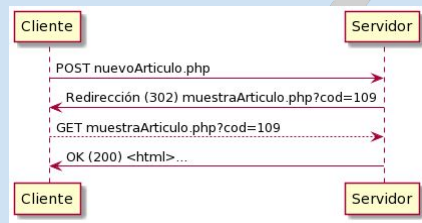
18

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Semántica de las peticiones HTTP

- Conviene respetar la semántica de las peticiones
 - **GET** para solicitar información (solo consulta)
 - **POST** para enviar información (modificación)
 - Problemas si una petición POST devuelve contenido html
 - ¿Qué pasa si se actualiza la página del navegador tras un POST?
 - ¿Qué pasa si el usuario pulsa el botón de ir atrás?
 - Solución: **Post/Redirect/Get** Pattern, PRG



19

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Semántica de las peticiones HTTP

- Tradicionalmente se han utilizado exclusivamente las peticiones GET/POST
 - Es fácil que se complique la comprensión y gestión de las urls
 - <http://host/app/consultaCliente.jsp?id=12>
 - <http://host/app/clientes.php?id=12&op=modifica>
 - <http://host/app/clientes/borrar.asp?id=12>
- En la actualidad es habitual el diseño RESTful de las URLs
 - **Enrutamiento** de URLs a módulos de código
 - Utilizar *nombres* en la urls para identificar **entidades** (recursos)
 - Semántica de métodos http para identificar **operaciones**
 - POST <http://host/app/clientes> // crear un cliente
 - GET <http://host/app/clientes> // listar todos los clientes
 - GET <http://host/app/clientes/12> // mostrar un cliente con id específico
 - PUT <http://host/app/clientes/12> // modificar un cliente
 - DELETE <http://host/app/clientes/12> // borrar un cliente

20

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Procesamiento en el lado del servidor

(Server-Side scripting)

Ejecución de código en el servidor

- Modalidades

- Ejecución de un **programa externo** en el propio sistema operativo del servidor. e.g. ejecutable, script bash, script perl, python, etc.
- Utilización de algún **módulo específico** del servidor para procesar el código existente en el recurso. e.g. módulos para ejecución de código php, python, perl en documentos web
- Reenvío de la petición a un “**recurso**” en un **servidor de aplicaciones** que la procese. e.g. método de un objeto en *servidor de servlets de Java*

Common Gateway Interface (CGI)

Especificación para intercambio de información entre el servidor web y un programa externo

Procedimiento:

- El servidor localiza el programa a partir de la url
- Lo ejecuta y le pasa la información:
 - **variables de entorno** (GET, cabecera HTTP, ...)
 - **entrada estándar**, e.g. `std::cin` (POST)
- Recoge su **salida estándar** (e.g. `std::cout`) y la pasa a la respuesta HTTP

Common Gateway Interface

Ejemplo de programa CGI en shell unix

<http://www.ujaen.es/cgi-bin/dimetuip.sh?user=jrbalsas>

```
#!/bin/bash

echo "Content-type: text/plain"
echo ""
echo "Tu ip es $REMOTE_ADDR"
echo "Me has pasado estos datos usando GET: $QUERY_STRING"
```

Inconvenientes:

- Recursos necesarios para cada llamada (duplicados)
- En servidores con una carga alta se consumen excesivos recursos de memoria y cpu

Common Gateway Interface (CGI)

Algunas variables de entorno

Variable	Descripción
HTTP_USER_AGENT	Identificación del navegador que hace la petición
SERVER_PROTOCOL	Versión HTTP de la petición. e.g 1.1
SERVER_PORT	Puerto por el que se ha recibido la petición. e.g. 80, 433
REQUEST_METHOD	Tipo de petición HTTP. e.g. GET, POST, PUT, DELETE, etc.
CONTENT_TYPE	Tipo de datos recibidos. e.g. text/html
CONTENT_LENGTH	Tamaño del bloque de datos recibido
SCRIPT_NAME	Ruta completa del recurso solicitado. e.g. /dir/search.do
PATH_INFO	Ruta posterior al recurso hasta el carácter ?
QUERY_STRING	Cadena posterior a carácter ? en la url. e.g word=latex&page=3
REMOTE_ADDR	Dirección ip del cliente

Módulo de servidor para ejecución de código

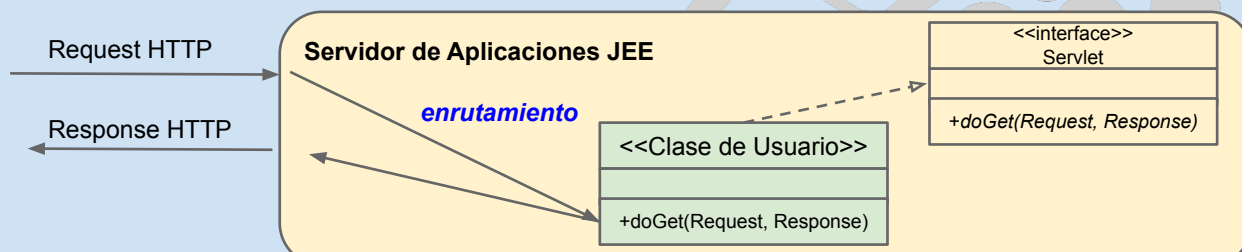
- El servidor dispone de extensiones para ejecutar código *incrustado* en documentos html
 - Ejemplos: php, perl, asp, scriptlets
- Mejor tiempo de respuesta
- Ejecución ligada al servidor web físico
- Ejemplo:

- <http://www.ujaen.es/util/consultatuip.php?user=jrbalsas>

```
<html>
  <head><title>Tu dirección IP</title></head>
  <body>
    Tu dirección IP es: <?php echo $_SERVER['REMOTE_ADDR']; ?>
    Me has pasado estos datos mediante GET:
    <?php foreach ($_GET as $variable=>$valor) {
      echo "$variable: $valor <br/>";
    };?>
  </body>
</html>
```

Servidor de Aplicaciones

- Sistema que atiende peticiones http en el servidor y las distribuye (enruta) a recursos específicos de una aplicación
- **Funciones**
 - Inicialización de los módulos del usuario, e.g. creación de objetos
 - Servicios a módulos, e.g. gestión conexiones, acceso a base de datos, mensajería, balanceo de carga, etc.
- **Ejemplos:**
 - Jakarta EE, MS .NET, php Zend Server, NodeJS, Django, Rails, ...



27

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Procesamiento en el lado del cliente

(Client-Side scripting)

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Funciones del navegador web

- **Conexión con servidor** (http/s, ftp, ...)
- **Recuperar y procesar información:**
 - **Procesar HTML y recursos referenciados:** imágenes, archivos javascript, archivos de hojas de estilo, audio, vídeo, etc.
 - Construir y manipular el DOM (Document Object Model)
 - Visualizar el DOM aplicando estilos (CSS)
 - **Ejecutar código en cliente** (Javascript) y permitir acceso “controlado” a recursos del navegador y sistema operativo: Impresora, GPS, etc.
 - **Plugins** especializados para visualizar/procesar otro tipo de datos. e.g. visualizar pdf, reproducir audio, ejecutar un módulo flash, un applet de java, etc.
- **Interaccionar con el usuario:**
 - Atender a eventos
 - Recoger y enviar la información que el usuario introduce a través de elementos HTML de formulario

29

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de ejecución en el cliente

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <style> #userip { color: red; }</style>
    <script>
      function obtenerIP () {
        fetch( 'https://webhost/webservice/ipcheck' )
          .then( response => response.text() )
          .then( ip => {

            let ipElement = document.getElementById('userip');
            ipElement.textContent = ip;

          })
          .catch( () => {
            console.log( "Error de conexión" );
          });
      };
    </script>
  </head>
  <body>
    
    <button onclick='obtenerIP()'>Consulta tu IP</button>
    Tu dirección ip es: <output id='userip'></output>
  </body>
</html>
```

El navegador utiliza un procesador diferente para cada tipo de contenido distinto:

- html
- css
- javascript
- imágenes
- ...

30

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Elementos de la interfaz web

- Una página web muestra/procesa diferentes contenidos descargados:

- Documento HTML
- Documentos de estilo asociados (.css)
- Documentos de código asociados (.js)
- Otros recursos referenciados: imágenes, audios, contenidos para plugins específicos, ...

Petición

```
GET /index.html HTTP/1.1
Accept: text/html
Host: www.ujaen.es
```

- ¿Cómo procesarlos?

→ Negociación contenidos

- El campo "Accept" de la petición HTTP indica los formatos de contenido esperados o aceptados
- El campo "Content-Type" de la respuesta HTTP indica al navegador cómo procesar la información.
- Ambos utilizan tipos **MIME** (*Multipurpose Internet Mail Extensions*):
 - [text/html](#), [image/jpeg](#), [image/png](#), [video/webm](#), [application/pdf](#), etc.

Respuesta

```
HTTP/1.1 200 OK

Content-Length: 25398
Content-Type: text/html; charset=utf-8

<!DOCTYPE html> <html><head>...
```

Motores de renderizado HTML

- Bibliotecas para gestionar y visualizar el *DOM* y recursos del navegador
- Alta complejidad... millones de líneas de código
- Pueden ser independientes del navegador
- Necesidad de documentos HTML bien definidos y uso de estándares para correcta visualización
- Diferencias en grado de implementación de novedades
 - <http://caniuse.com> <http://html5test.com>
- Algunos motores habituales
 - Trident: Internet Explorer, Edge
 - WebKit: Chrome, Safari
 - Presto: Opera
 - Gecko/Quantum: Firefox, LibreWolf, Tor browser...
 - Blink: Chrome, Opera, Brave, Vivaldi, Edge



Bibliografía

- Deitel, Paul J, Abbey Deitel, and Harvey M. Deitel. *Internet & World Wide Web : How to Program*. 5th ed. Pearson, 2012. [[Recurso electrónico UJA](#)]
- Pilgrim, Mark. *HTML5, Up and Running*. O'Reilly-Google press, 2010. [[Recurso electrónico UJA](#)]

Lecturas Complementarias

- Sebesta, Robert W. *Programming the World Wide Web*, 7th edition. Addison-Wesley, 2012.
- *Web development* (n.d.) Wikipedia. Retrieved January, 2020 from http://en.wikipedia.org/wiki/Web_development
- *Rich Internet Application* (n.d.) Wikipedia: Retrieved January, 2020 from http://en.wikipedia.org/wiki/Rich_Internet_application